

DESIGN DOCUMENT

AI Rewrite

High-Level Design — a free, client-only Grammarly alternative built on Google Gemini's free API tier

Version	1.0
Date	August 4, 2026
Author	Somil Merugawar
Repository	AI Rewrite (browser-extension · desktop-autohotkey · git-commit-tool)
Status	Active — v1.0.0 shipped, MIT licensed

1. Overview

1.1 Problem statement

Writing-assistance tools like Grammarly gate their most useful features (rewrites, tone changes, advanced grammar fixes) behind a paid subscription. Most users only need this occasionally, and modern LLMs can do the same underlying job — given a piece of text and an instruction, return a corrected/rewritten version — at effectively zero cost via free API tiers.

1.2 Goals

- Let a user fix, rewrite, or change the tone of any selected text, in any application (browser or native), with a single action (menu click or hotkey).
- Cost the user **\$0** — no subscription, no backend to operate or pay for, no credit card required for the underlying AI provider.
- Keep the user's text and API key fully client-side — never routed through any server owned by this project.
- Stay simple enough that a non-expert can read the entire source and understand what it does.

1.3 Non-goals

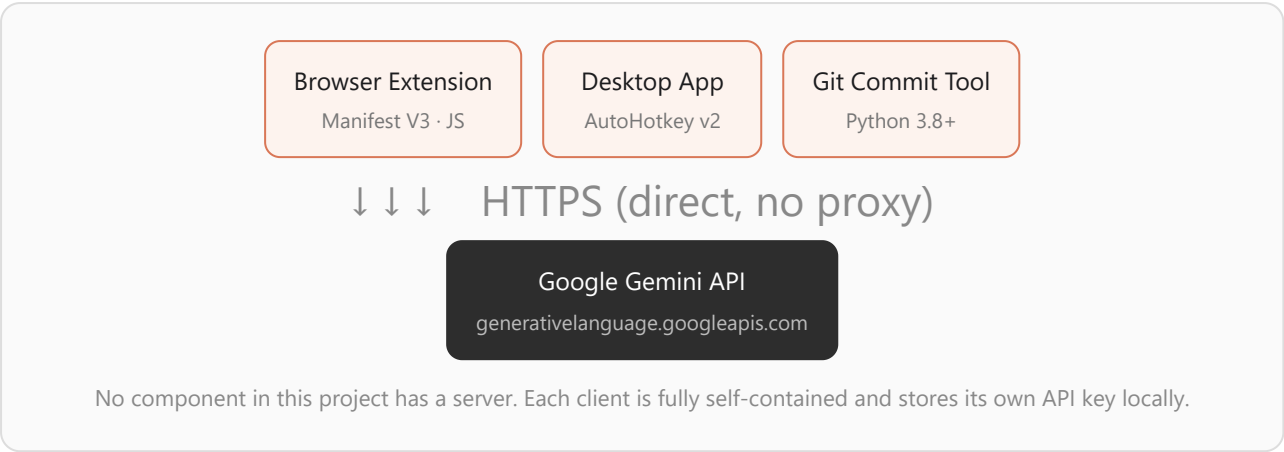
- Not a real-time, as-you-type grammar checker (like Grammarly's underline-as-you-go) — it is an on-demand, select-and-trigger tool.
- Not a hosted/managed product — there is no account system, no telemetry, no multi-user backend.
- Not cross-platform for the desktop app in v1 — Windows only (AutoHotkey is Windows-native); macOS/Linux desktop support is a roadmap item, not a current goal.

Key design decision: there is no backend server anywhere in this system. Every client (browser extension, desktop app, CLI tool) talks directly to Google's Gemini API over HTTPS using a key the user supplies and stores locally. This single decision drives most of the architecture: no auth system, no hosting cost, no server-side attack surface, and a privacy story that reduces to "your key, your data, straight to Google."

2. System Architecture

2.1 Component map

The project ships three independent client applications and one packaging tool. They share no runtime code or process — each is built with the technology best suited to where it needs to run, and each independently implements the same "call Gemini, get text back" logic.



2.2 Component responsibilities

Component	Runs in	Trigger	Responsibility
browser-extension/	Chrome / Edge (MV3)	Right-click context menu on selected text	Rewrite text inside any web page — <code><textarea></code> , <code><input></code> , or <code>contenteditable</code> (Gmail, Docs, etc.)
desktop-autohotkey/	Windows, any app	Global hotkey (<code>Ctrl+Alt+R/I/F/C/S</code>)	Rewrite text system-wide via simulated copy/paste, independent of the target app
git-commit-tool/	Terminal (Python)	Manual run in place of <code>git commit</code>	Generate a Conventional Commits message from the staged diff
installer/	Build-time + Windows setup	N/A (packaging)	Compiles the desktop app and git tool into standalone exes and bundles both into one guided installer/uninstaller

2.3 Common request pattern

All three clients implement the identical integration contract against the Gemini `generateContent` endpoint, independently, in their native language:

```
POST https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent
Header: x-goog-api-key: {user's key}
Body:  { "contents": [{ "role": "user",
                        "parts": [{ "text": "{instruction}\n\nTEXT:\n{selectedText}" }] }] }

Response text extracted from:
  candidates[0].content.parts[0].text
```

Default model: `gemini-3.1-flash-lite` (free tier, ~1,500 requests/day); the browser extension additionally offers `gemini-3.6-flash` as a selectable, stronger alternative.

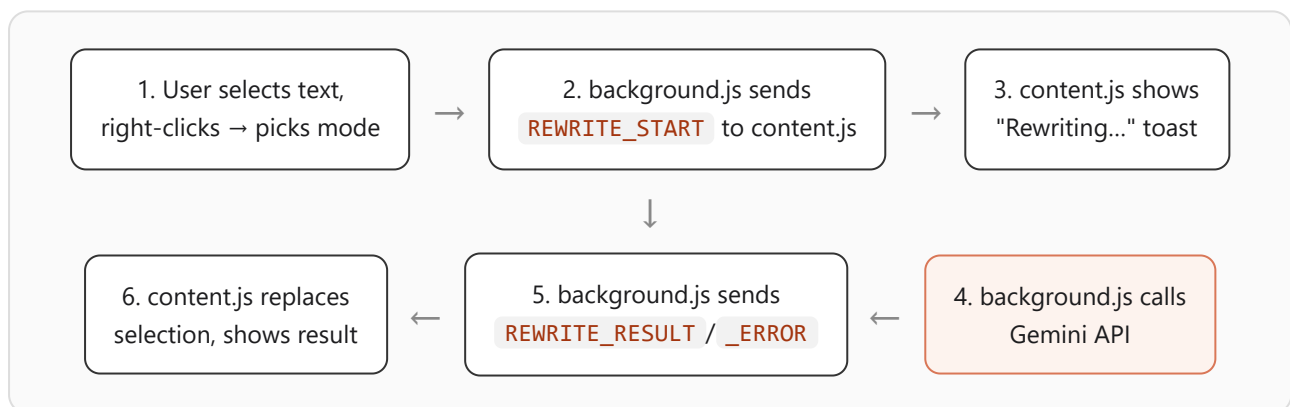
Design trade-off: the prompt/instruction strings for the five rewrite modes (Fix Grammar, Improve Writing, Make Formal, Make Casual, Make Shorter) are duplicated in `background.js` and `AI_Rewrite.ahk` rather than shared from a single source. This is a direct consequence of having no shared runtime between a browser extension and a native Windows script — the duplication is accepted in exchange for zero build tooling and zero cross-component dependencies.

3. Detailed Design — Browser Extension

3.1 Structure

File	Role
<code>manifest.json</code>	MV3 manifest — permissions (<code>contextMenus</code> , <code>activeTab</code> , <code>scripting</code> , <code>storage</code>), host permission scoped to Gemini's API domain only
<code>background.js</code>	Service worker — builds the context menu, handles clicks, calls Gemini
<code>content.js</code>	Injected into every page — tracks the current selection, replaces it in place, shows a status toast
<code>popup.html</code> / <code>popup.js</code>	Settings UI — API key + model, persisted to <code>chrome.storage.sync</code>

3.2 Rewrite sequence



Why a separate content script tracks selection: the DOM selection can be lost the moment the native browser context menu opens (behavior varies by site), so `content.js` listens to `selectionchange` continuously and caches the last non-collapsed selection — either an `<input>` / `<textarea>` range (by `selectionStart` / `selectionEnd`) or a DOM `Range` for `contenteditable` content — so it can be replaced correctly once the async API call resolves, regardless of what focus/selection state exists by then.

For `contenteditable` targets, after replacing the range's contents an `input` event is dispatched on the editable root so that React-based editors (Gmail, Google Docs) detect the DOM mutation and update their own internal state — without this, the visible text would change but the app's model of the document would not.

3.3 Failure handling

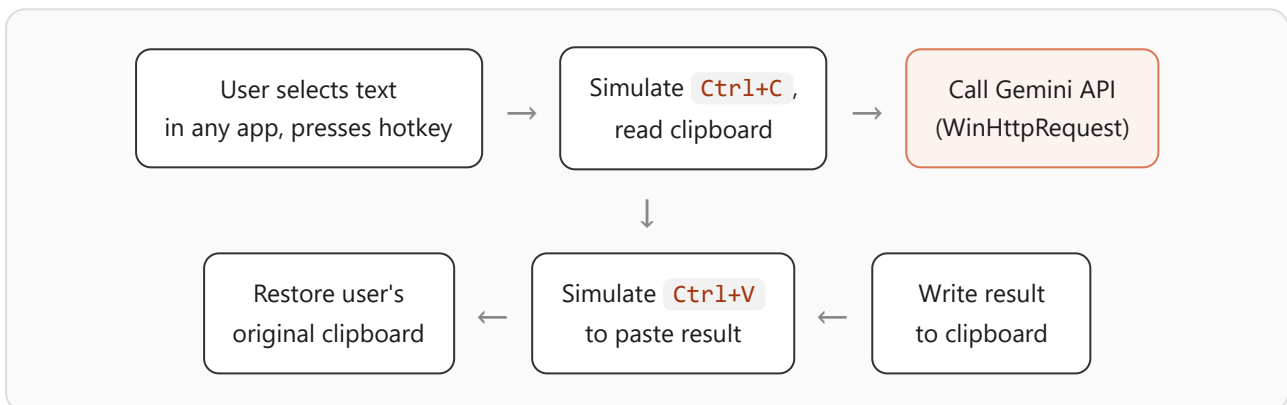
If no cached selection exists when the result arrives (edge case), the tool falls back to copying the rewritten text to the clipboard and showing an error toast rather than silently failing. Errors are shown for 8 seconds (vs. 3 seconds for success) since they're the messages a user actually needs time to read.

4. Detailed Design — Desktop App (AutoHotkey)

4.1 Why AutoHotkey

The desktop app's core requirement — reliably intercepting a global hotkey and simulating keystrokes in *any* foreground Windows application, regardless of that app's own extensibility — is exactly AutoHotkey's purpose-built niche. A general-purpose language would need extra OS-hook libraries to do the same job.

4.2 Rewrite sequence



Because there is no accessibility-API integration with arbitrary third-party apps, text retrieval/replacement is done the only universally reliable way on Windows: simulated `Ctrl+C` / `Ctrl+V` via `Send()`. The user's pre-existing clipboard contents are snapshotted before the operation and restored afterward so nothing they had previously copied is lost.

4.3 Hand-rolled JSON

AutoHotkey v2 has no built-in JSON library, so both directions of the Gemini exchange are handled manually:

- **Request:** the prompt string is escaped by hand (backslashes, quotes, newlines) and interpolated into a literal JSON body string.
- **Response:** rather than a general parser, the code locates the literal `"text":` field and walks forward counting consecutive backslashes before each `"` it finds, to correctly distinguish an escaped quote from the real closing quote of the string.

Maintenance note: this hand-rolled extraction is coupled to Gemini's current response shape (`candidates[0].content.parts[0].text`). If the API response format changes, this is the first code path to break and to check.

4.4 Configuration & first run

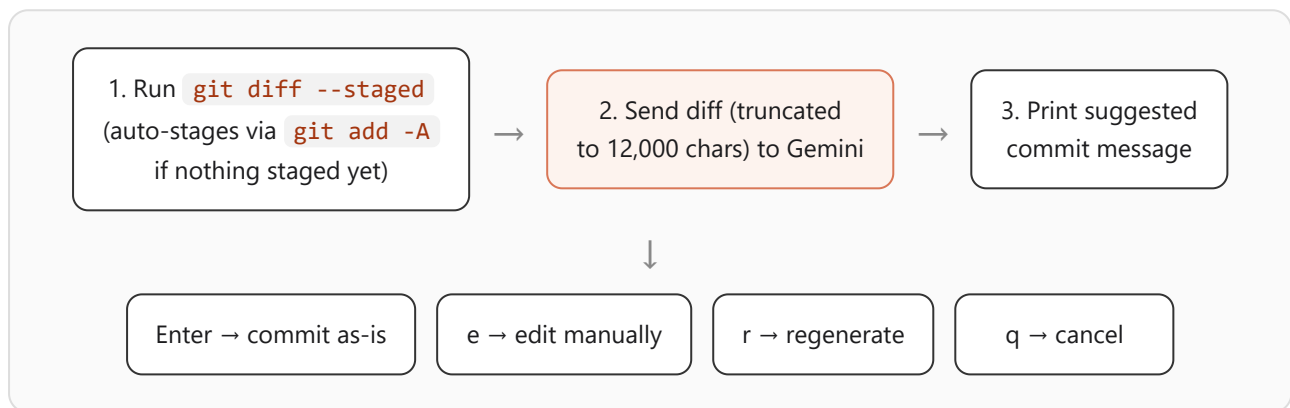
On first launch, if no `config.ini` exists next to the script, a small AHK GUI prompts for the Gemini API key and writes it (plus the default model) to `config.ini`. The app then minimizes to the system tray; the tray menu lists all five hotkeys (disabled, informational only) and an "Update API Key" action.

5. Detailed Design — Git Commit Tool

5.1 Purpose

A CLI utility that replaces `git commit -m "..."` with an AI-suggested Conventional Commits message generated from the actual staged diff, reviewed and confirmed by the user before anything is committed.

5.2 Flow



5.3 API key resolution order

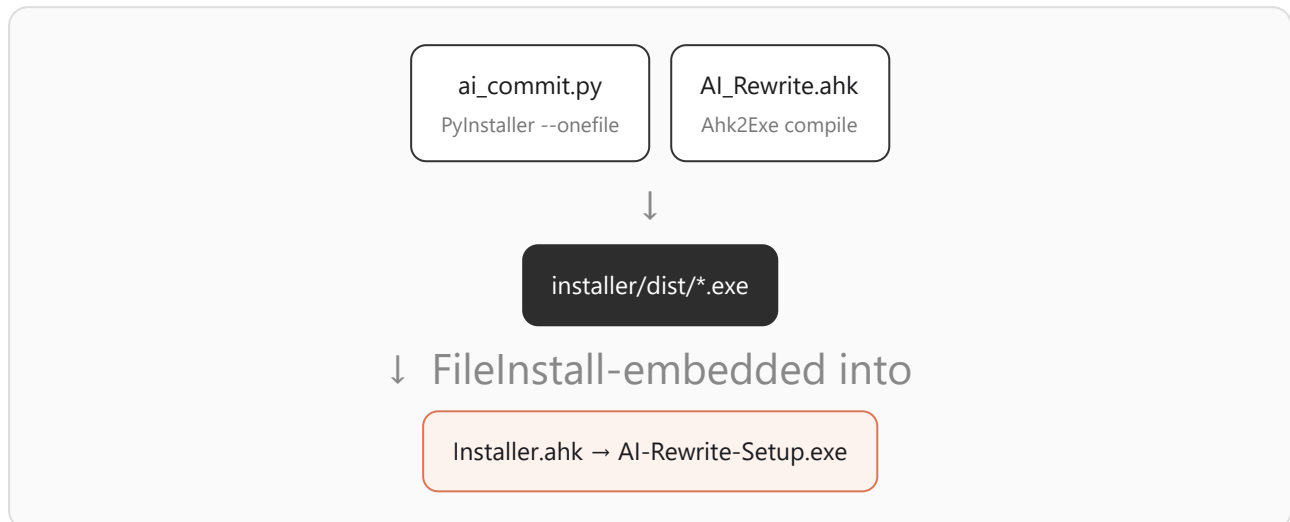
1. `GEMINI_API_KEY` environment variable (recommended — works across all repos)
2. `~/.ai-commit/config.ini` (written automatically after the first interactive prompt)
3. Interactive prompt on first run if neither is found, then persisted to the config file

5.4 Safety properties

- Only ever reads *staged* changes — matches native `git commit` semantics and never commits unstaged work unexpectedly.
- A plain run always stops at the confirmation prompt — nothing is committed without explicit user action (Enter/e/r/q).
- Does not scan for secrets in the diff — the README explicitly calls this out as the user's responsibility before confirming.

6. Installer & Packaging

6.1 Build pipeline



`installer/build.ps1` orchestrates all three compilation steps in order — the two component exes must exist before `Installer.ahk` is compiled, since it embeds them via `FileInstall` at compile time.

6.2 Installer behavior

- Single GUI lets the user opt in/out of each component (AI Rewrite / AI Commit) independently, choose an install directory, and optionally pre-fill a Gemini API key for both tools at once.
- Writes a standard `HKCU\...\Uninstall` registry entry so the app is listed in Windows' "Add or Remove Programs".
- Optionally creates a Startup-folder shortcut (auto-launch on login) and adds the install directory to the user's `PATH` (so `ai-commit` works from any terminal).
- The same exe doubles as the uninstaller (invoked with `/uninstall`); since a running exe can't delete its own containing folder, final cleanup is handed off to a detached `cmd /c rmdir` after the process exits.

7. Cross-Cutting Concerns

7.1 Security & privacy

Concern	Design
API key storage	Local only — <code>chrome.storage.sync</code> (browser), plaintext <code>config.ini</code> (desktop app), env var or <code>~/.ai-commit/config.ini</code> (git tool). Never transmitted anywhere except the Gemini <code>x-goog-api-key</code> header.
Text/diff content	Sent only to <code>generativelanguage.googleapis.com</code> over HTTPS; not logged, stored, or sent to any server owned by this project (there is none).
Browser host permissions	Scoped narrowly to Gemini's API domain only — not a broad <code><all_urls></code> network grant (that permission is used only for the content-script injection, not network access).
Vulnerability reporting	Security issues are requested to be filed as a flagged issue rather than a public PR with exploit details (see CONTRIBUTING.md).

7.2 Cost model

Entirely free to run: Gemini's free tier (Flash / Flash-Lite models) covers roughly 1,500 requests/day per key, far beyond typical personal usage, with no credit card requirement. There is no hosting cost because there is no hosted component.

7.3 Reliability & error handling

- All three clients surface API errors (non-2xx status, missing response text) directly to the user rather than failing silently — as a toast (browser), a tray notification (desktop), or printed CLI output (git tool).
- The desktop app always restores the user's original clipboard after an operation, success or failure, so a failed rewrite never leaves the user's clipboard corrupted.
- The git tool never leaves a partially-typed commit: any failure before the final confirmation simply exits without committing.

7.4 Extensibility

Adding a new rewrite mode means adding one entry to the `MODES` array in `background.js` and one corresponding hotkey binding in `AI_Rewrite.ahk` — there is intentionally no shared config file between the two, per the "no new dependencies / stay lightweight" principle in CONTRIBUTING.md.

8. Roadmap

From the project README, not yet implemented:

- Firefox support for the browser extension
- macOS desktop version (would require a different toolchain, e.g. Hammerspoon, since AutoHotkey is Windows-only)
- Custom user-defined rewrite modes
- Inline diff preview before replacing text

AI Rewrite — High-Level Design v1.0 — generated from repository analysis (README.md, CONTRIBUTING.md, and source in browser-extension/, desktop-autohotkey/, git-commit-tool/, installer/). MIT licensed.